Why Irradix Never Contains `101`

A stronger characterization, a proof, and exact algorithms

Analysis of the `irradix` repository

September 7, 2026

Abstract

The exact integer-quotient version of this repository's golden-ratio encoding has a complete and simple description: its positive codewords are exactly the binary strings beginning with `1` in which every run of zeros between successive ones has even length. Thus `101` is forbidden, as are `10001`, `1000001`, and all longer odd-zero gaps. The proof comes from a contracting, sign-reversing map whose fixed point is φ^{-2}. We derive the Fibonacci count of codewords, an exact integer-only encoder, and the practical consequences for packing. Finite-precision implementations are distinguished from the mathematical encoding throughout.

1 The precise object being proved

Put $\varphi = (1+\sqrt{5})/2$. For a nonnegative integer n, the code's modulo operation, interpreted in exact arithmetic, performs

$$q = \lfloor n/\varphi \rfloor, \qquad d = \lfloor n - \varphi q \rfloor, \qquad \mathcal{E}(n) = \mathcal{E}(q)\, d. \tag{1}$$

Use the empty word for $\mathcal{E}(0)$ while recursing. The public representation of zero is the special string `0`. For positive inputs the recursion terminates because $0 \le q < n$, and $d \in \{0,1\}$ because $0 \le n - \varphi q < \varphi < 2$.

This is *not* the ordinary positional base-φ expansion. For example, the repository represents 2 by `10`, whose ordinary positional value would be φ, not 2. The correct inverse is

$$A(q) = \lceil \varphi q \rceil, \qquad n = A(q) + d. \tag{2}$$

Indeed, since n is an integer, $\lfloor n - \varphi q \rfloor = n - \lceil \varphi q \rceil$. Decoding therefore applies a ceiling at *each* digit. One ceiling around a sum of powers is not equivalent.

Definition 1.1. A digit d is admissible after a prefix with decoded value q if $\lfloor (A(q)+d)/\varphi \rfloor = q$.

Appending `0` is always admissible. For $q > 0$, φq is irrational; write $t(q) = \{\varphi q\} \in (0,1)$. Then

$$\texttt{1} \text{ is admissible after } q \quad \Longleftrightarrow \quad t(q) > 2 - \varphi. \tag{3}$$

To see this, $A(q) = \varphi q + 1 - t(q)$, so the upper quotient bound is $\varphi q + 2 - t(q) < \varphi(q+1)$. The lower quotient bound is automatic. At $q = 0$, the first `1` is admissible and produces $q = 1$.

2 The phase identity: where the golden ratio enters

Set

$$a = \varphi - 1 = \varphi^{-1}, \qquad c = 1 - a = 2 - \varphi = a^2.$$

Thus $0 < a < 1$, $a^2 + a = 1$, and $c \approx 0.381966$. The threshold in (3) is exactly c.

Lemma 2.1 (Effect of appending a zero)**.** *For every positive integer q,*

$$t(A(q)) = a(1 - t(q)).$$

Equivalently, the zero transition $T(t) = a(1 - t)$ satisfies

$$T(t) - c = -a(t - c). \tag{4}$$

Proof. Let $m = \lfloor \varphi q \rfloor$, so $A(q) = m + 1$ and $\varphi q = m + t$. Using $\varphi^2 = \varphi + 1$ gives

$$\varphi A(q) = (m + q + 1) + a(1 - t).$$

The parenthesized expression is an integer and $0 < a(1 - t) < 1$, so the final term is the fractional part. Finally, $a(1 - c) = c$ follows from $a^2 + a = 1$, proving (4). □

Lemma 2.2 (Effect of appending a one)**.** *After any admissible `1`, the phase lies in $[a, 1)$, hence strictly above c.*

Proof. The initial `1` has phase $t(1) = \varphi - 1 = a > c$. For a positive prefix, an admissible `1` means $t > c$. From the previous calculation,

$$\varphi(A(q) + 1) = (m + q + 2) + a(2 - t).$$

Here $a < a(2 - t) < 1$: the upper bound is precisely $t > c$, since $a(2 - c) = 1$. Thus the new fractional part is $a(2 - t)$. □

Theorem 2.3 (The even-gap characterization)**.** *A nonempty binary word is $\mathcal{E}(n)$ for some positive integer n if and only if it starts with `1` and every zero run between successive `1`s has even length. Trailing zeros may have either parity.*

Proof. Immediately after a `1`, let the phase be $t_0 > c$. After j zeros, repeated use of (4) gives

$$t_j - c = (-a)^j (t_0 - c). \tag{5}$$

Because $a > 0$, this is positive for even j and negative for odd j. By (3), the next `1` is admissible exactly when j is even. This proves necessity.

Conversely, start with `1` and read a word satisfying the stated condition. Every zero is admissible. Before each subsequent one the number of zeros since the previous one is even, so (5) makes that one admissible as well. Each inverse step has exactly the quotient and remainder in (1); reversing the steps therefore recovers the word from its decoded integer. This proves sufficiency. No further digit needs to follow trailing zeros, so they have no parity restriction. □

Corollary 2.4. *For every $j \geq 0$, no exact codeword contains $1\,0^{2j+1}\,1$. In particular, `101` never occurs.*

A short version of the proof. A `1` puts the phase above c; a `0` reflects it below c; a following `1` would require it to be above c. That contradiction excludes `101`. Equation (4) is the exact reason $\varphi^2 = \varphi + 1$ matters. Irrationality alone does not imply this property: for base $\sqrt{2}$, the same quotient algorithm maps 4 to `101`. We do not claim here that φ is the only real base with this exclusion.

3 Fibonacci counts and an exact arithmetic replacement

The permitted suffixes after the initial `1` are precisely

$$(\mathtt{1} \mid \mathtt{00})^*(\varepsilon \mid \mathtt{0}).$$

Equivalently, remember whether the zero count since the last one is even or odd. From even, a one stays even and a zero goes odd; from odd, only a zero is permitted, returning to even. This is the classical *even shift* language [1]. The equivalence between the repository's quotient algorithm and that language was established above; the language itself is not new.

Let $F_0 = 0$, $F_1 = 1$, and $F_{j+2} = F_{j+1} + F_j$. There are exactly F_{k+1} positive codewords of length k. Indeed, if C_r counts suffixes of length r from the even state, then $C_0 = 1$, $C_1 = 2$, and $C_r = C_{r-1} + C_{r-2}$. Consequently

$$|\mathcal{E}(n)| = k \quad \Longleftrightarrow \quad F_{k+2} - 1 \le n \le F_{k+3} - 2. \tag{6}$$

Here is an independent verification that these counts occur in integer order. The quotient $\lfloor n/\varphi \rfloor$ is nondecreasing in n; each quotient has either a zero child or a zero child followed by a one child. Induction therefore shows that the binary values of $\mathcal{E}(n)$ are strictly increasing. Summing the length counts yields (6).

Length k	Smallest n	Largest n	Codewords
1	1	1	1
2	2	3	2
3	4	6	3
4	7	11	5
5	12	19	8
6	20	32	13

3.1 A direct Fibonacci formula

For a valid word $w = 1d_{k-2} \cdots d_0$ of length k,

$$\boxed{D(w) = F_{k+2} - 1 + \sum_{j=0}^{k-2} d_j F_{j+1}.} \tag{7}$$

To prove this, rank the length-k words lexicographically, starting at rank zero. Whenever a one is chosen with r positions remaining, the skipped zero branch contains exactly F_r completions. At an odd state a zero is forced and skips nothing. Thus the rank is the sum in (7); add the smallest integer of that length. For example, `1001` decodes to $F_6 - 1 + F_1 = 7 + 1 = 8$.

Encoding reverses this ranking procedure: locate k using (6), subtract $F_{k+2} - 1$, and at each even state choose zero if the remaining rank is below F_r, otherwise choose one and subtract F_r. At an odd state output the forced zero. This uses $O(k)$ integer additions, comparisons and subtractions, with $O(k)$ Fibonacci table entries. This is an arithmetic-operation count; large-integer operations are not constant-time.

3.2 An independent square-root reference

An exact version of the original algorithm is also available from integer square roots:

$$\lfloor n/\varphi \rfloor = \frac{\operatorname{isqrt}(5n^2) - n}{2} \text{ rounded down,}$$

$$A(0) = 0, \qquad A(q) = \left\lfloor \frac{q + \operatorname{isqrt}(5q^2)}{2} \right\rfloor + 1 \quad (q > 0).$$

These formulas follow from $\varphi = (1 + \sqrt{5})/2$ and the fact that $\sqrt{5}\, q$ is irrational for positive integer q. They provide an independent reference against which the faster Fibonacci implementation can be tested, without choosing floating-point precision.

4 Consequences for packing

The expansion is structural. The Fibonacci counts imply

$$|\mathcal{E}(n)| = \log_\varphi n + O(1) = 1.440420\ldots \log_2 n + O(1).$$

This is not compression of unrestricted integers. Its redundancy buys a constrained bit language. The even-shift transition matrix $\left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ has largest eigenvalue φ, explaining the same constant combinatorially.

If only `101` must be forbidden, a denser code is possible. Allowing all `101`-free words discards the extra odd-gap restrictions. Its three-state prefix automaton has characteristic polynomial $\lambda^3 - 2\lambda^2 + \lambda - 1$ and largest root $\lambda \approx 1.754878$. Enumerative block coding can approach $1/\log_2 \lambda \approx 1.232$ output bits per input bit before framing. This is a different code and requires a separate boundary design.

Delimiters still need boundary handling. The absence of `101` inside each payload does not alone prove that joining payloads with `101` is safe: a payload ending in `10` can create an earlier crossing occurrence. The repository uses doubling and a suffix repair. A simple alternative prototype is

$$\mathcal{E}(n+1)\ \texttt{00}\ \texttt{101}.$$

The two guard zeros prevent a premature crossing delimiter; decoding splits at `101`, removes exactly two zeros, validates the payload, and subtracts one. It costs five framing bits per value, including a terminal delimiter. It is a new format, not a compatible replacement. Neither this delimiter nor the even-gap rule is a checksum.

Length-first packing remains the useful large-integer variant. Writing raw binary payloads and encoding only their lengths changes the per-value cost to $B + O(\log B)$ for a B-bit integer. That explains the repository's large-integer advantage over VByte; direct Irradix still pays approximately $1.44B$. Elias delta and Fibonacci coding are relevant established baselines [2, 3]. For a deterministic sample of 1,000 integers of 50–100 decimal digits, the reproduced storage counts were:

Method	Stored bits, including byte rounding
Raw payload only (not a framed code)	248,161
Direct Irradix	361,216
Length-first Irradix	264,240
Elias delta, applied to $n+1$	263,072
VByte	286,984

These are sample-dependent sizes, not an entropy bound or a throughput comparison. If many lengths repeat, block widths or compressed length runs are further opportunities.

The prime-density claim needs a different baseline. Interpreting codewords as binary gives a sparse, biased integer set. Among $n = 1, \ldots, 1000$ the repository's 94 mapped primes are reproducible, but only 382 mapped values are odd. Asymptotically the odd proportion is $1 - 1/\varphi = \varphi^{-2}$: each quotient contributes one zero-ending child, and any second child ends in one. Prime density scales roughly with inverse logarithmic magnitude, not inverse magnitude; dividing 16.8% by eight is unjustified. A parity-adjusted logarithmic heuristic gives about 107 primes for this sample, rather than 21. This is a heuristic, not a prime-distribution theorem; other residue biases remain.

5 Verification and scope

The mathematical theorem concerns *exact* quotient and ceiling operations. The repository's fixed-precision `mpmath` code and its C++ floating-point version are approximations. The reproduced run finds round-trip failures near 10^{100} at the configured precision of 100 decimal digits. Such numerical failures do not contradict the theorem, but they matter for an implementation claiming arbitrary-size integers.

The accompanying Python reference independently checks the original integer-square-root construction and the Fibonacci construction on 100,001 values; checks 500 Fibonacci length thresholds; and tests 4,369 short sequences per experimental codec. The machine-checked Lean companion formalizes the exact ceiling-step model, the phase transition argument, and the forbidden odd-gap result. Its build instructions and precise theorem inventory are in `research/lean/README.md`. The Python implementation and the packing formats themselves are not formally verified by that Lean proof. Numerical tests are supplementary evidence, not a substitute for the all-integers theorem.

References

[1] D. Glasscock, J. Moreira, and F. K. Richter, *Additive and geometric transversality of fractal sets in the integers*, Journal of the London Mathematical Society (2024). doi:10.1112/jlms.12902. The even shift is discussed as a binary digit restriction.

[2] A. Apostolico and A. S. Fraenkel, *Robust Transmission of Unbounded Strings Using Fibonacci Representations*, Purdue technical report 85-545 (1985). https://docs.lib.purdue.edu/cstech/464/.

[3] D. A. Lelewer and D. S. Hirschberg, *Data Compression*, section 3, universal codes. https://ics.uci.edu/~dhirschb/pubs/DC-Sec3.html.